Date:
Page:

UNIT-1
BASICS OF COMPUTING & PERFORMANCE METRICS

Computer Architecture
- set of rules and methods that describe the functionality, organization and implementation of computer systems.
- describes relationships b/w software, hardware components, memory, processors and I/O systems.

Technology Trends

① Architectural Trends

a) From single-core to multi-core
- Performance ↑ by processor clock speed ↑ but caused excessive heat & high power consumption
- Shifted to multi-core processors where CPU contains multiple execution cores enabling parallel execution & faster performance

b) Parallel & Distributed Computing
- Parallel - divides work into multiple smaller tasks that run simultaneously. Execution speed ↑
- Distributed - executes tasks across multiple network-connected computers to perform large-scale processing

c) Rise of Heterogeneous Computing
- Integrates CPU, GPU, TPU, FPGA and NPU (Neural Processing Unit)

(Central Processing Unit / Graphics Processing Unit / Tensor Processing Unit / Field-Programmable Gate Array)

② Architectural Design Trend

a) RISC vs CISC
- Reduced Instruction Set Computing - uses fewer, simple instructions

focusing on high speed & pipeline efficiency (ARM processor)
↳ Complex Instruction Set Computing (CISC) - supports complex instructions, enables compact program size (Intel x86 processors)

b) Multi Threading & Hyper-Threading
↳ Simultaneous Multi-threading allows one core to run multiple threads simultaneously. Resource utilization ↑
↳ Hyper-Threading enhance performance without extra physical cores

c) Cache Hierarchy Improvements
↳ Modern CPU - multi level cache memory (level 1-3) to store frequently used data. ↓ Memory Access Time

d) Virtualization Support
↳ Hardware assisted Virtualization Technologies (Intel VTx and AMD-V) enables multiple OS to run on single system

e) Power-Efficient Architecture
↳ Modern processors prioritize performance per watt using techniques like DVFS and Big.Little architecture.

③ Current Industry Trends

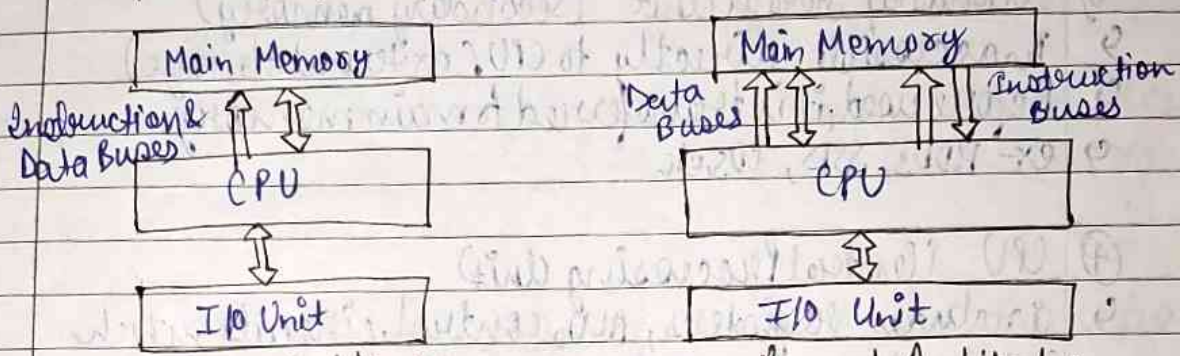
a) ARM-Based Architectures
↳ Advanced RISC Machines - low power consumption & high efficiency (Apple M1/M2/M3 processors)

b) Cloud and Data Center Architecture
↳ Modern data centers use scalable & redundant distributed architectures with virtualization & automation.
↳ Cloud platforms (AWS, Azure, etc) deliver on-demand computing resources using virtual machines & containers

c) AI-Driven Architectures
↳ specialized accelerators like GPU, TPU and NPU boost parallel processing & matrix operations used in deep learning training & inference

d) Edge and IoT Computing
- Edge - processes data near the source rather than sending to cloud (smart-city devices, real-time robotics, autonomous vehicles
e) Quantum Computing
- uses qubits (quantum bits) which can exist in multiple states simultaneously.
- enables extremely complex calculations used in cryptography, optimizations & scientif research.

Components of Computer Architecture



Von-Neumann Architecture

Harvard Architecture

Components: primarily- CPU, Memory & Peripherals linked by system bus (Address, Data and Control)

(1) Input Unit and associated peripherals
- receives information from input devices, translates it to machine language, then inserts it within computer system
ex- Keyboard, mouse, etc have hardware drivers to work in sync with rest of architecture

(2) Output unit and associated peripherals
- delivers computer process's results to the user
- ex- printing unit, display, speakers, etc.

Date:
Page:

(3) Storage Unit / Memory
- contains computer parts that store data
- Combines:
  a) Primary Storage Unit (main memory)
  - CPU has direct access to it.
  - used for storing info. and instructions during program execution. Two kinds:
    RAM - temporary; supplies info straight to CPU.
    ROM - permanent; contains pre-installed instructions; used to boot machine on startup
  b) Secondary Storage Unit (secondary memory)
  - Inaccessible directly to CPU (external storage)
  - Before used, it is transferred to main memory.
  - ex- HDDs, SSD, CDs etc

(4) CPU (Central Processing Unit)
- includes registers, ALU, control circuits which interpret and execute assembly language instructions
- interacts with all other parts to make sense of data & deliver necessary output.

Sub-components:

(i) Registers
- high-speed, purpose-built temporary memory devices
- rather than referring by address, they are accessed directly by CPU.
- Contain info, commands, addresses, intermediate processing results

(ii) ALU (Arithmetic Logic Unit)
- includes electrical circuitary that performs any arithmetic & logical processes on supplied data.
- Registers are used to retain data being processed

(iv) Control Unit
- collaborates with computer's input and output devices
- arrange data & instruction processing via comm" with ALU and registers
- Microprocessor is primary component that runs CPU additionally components

(5) Bootloader (in firmware)
- program executed by processor that retrieves operating system from disc and loads it into memory for processor

(6) Operating System
- offers set of tools for programs, allowing them to access screen, disc drivers and other elements of architecture

(7) Buses
- tangible collection of signal lines with linked purpose
- transfers info from one system to other.

Types:
- Data Bus - carries data among main memory, processor & I/O devices
- Data Address Bus - carries address of data from CPU to main memory
- Instruction Bus - carries instructions among main memory, proc, & I/O
- Instruction Address Bus - carries address of instructions from processor to main memory

(8) Interrupts
- exceptions or traps, methods for redirecting processor from running of current program to handle an occurrence
- event might be a malfunction or a I/O device has completed previous duty

Moore's law

- observation made by Gordon Moore (founder of Intel)
- number of transistors on a chip doubles approximat-ly every 18 months, leading to exponential growth in complexity of computer chips over time
- became a driving force behind development of more powerful and efficient CPUs.

Advantages:

1) Increased processing power
- enabled CPU's to deliver higher processing power, allowing more complex computations & tasks to be performed
2) Enhanced efficiency and speed
- led to faster CPU speeds and reduced power consumption increasing energy efficiency
3) Cost reductions and economies of scale
- reduced cost of manufacturing CPUs, making technology accessible.

Challenges & Limitations

1) Physical constraints & Heat Dissipation
- As transistor sizes shrink, physical limitations becomes challenging (to prevent overheating)
2) Diminishing Returns & Technological Roadblocks
- exponential nature faces limits as industry encounters diminishing returns in transistor scaling leading to technological roadblocks that slow down pace of CPU development.

Types of Computer Architecture

- Von-Neumann
- Harvard
- Instruction Set
- Micro Architecture
- System Design

① Von-Neumann	② Harvard
ancient, based on stored program computer concept	modern, based on Harvard Mark I relay based model
same physical memory addressed for data & instruction	Separate physical memory address for instructions and data
common bus for data & instruction transfer	Separate bus for data & instruction transfer
Two clock cycles required to execute single instruction	Single clock cycle used for single instruction
cheaper	costly
CPU can not access instruction & read/write at same time	CPU can access instructions and read/write at same time

③ Instruction Set Architecture

- Holds a collection of instructions that the processor renders and surmises.
- Consists of two instruction sets: RISC (Reduced Instruction Set Computer) and CISC (Complex Instruction Set Computer)
- enables versatile implementations of ISA, differing in features such as performance, physical size & monetary price

④ Micro Architecture

- structural design of microprocessor
- instruction set architecture holds a built-in processor.
- includes technologies used, resources and methods.
- logical form of all electronic elements and data pathways present in microprocessor.

Date:
Page:

(5) System Design
- defines a design that can serve user requirements like system architecture, computer modules with various interfaces & data management within a system
- overall system architecture based on requirements

Parallelism (Parallel Computing)
- use of multiple processing elements simultaneously for solving any problem
- Problems broken into instructions are solved concurrently as each resource that has been applied to work is working at same time
- saves time and money
- better use of hardware to solve larger problems

Types of Parallelism

(1) Bit-level Parallelism
- based on increasing processor's size.
- expands processor's word size, reducing the number of instructions needed for operations on larger data types

ex- 8-bit processor - for sum of two 16 bit integers requires 2 instructions (8-lower & 8 higher)
16 bit processor - uses one instruction for same

(2) Instruction Level Parallelism
- enables simultaneous execution of multiple instructions from a program.
- Modern processors use techniques like pipelining and superscalar execution to achieve this

③ Task-level Parallelism
- employs decomposition of task into subtasks and then allocating each of the subtask for execution
- Processor perform execution of subtasks concurrently

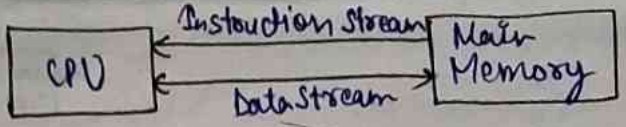
④ Data-level Parallelism
- concurrent execution of same task on multiple computing cores but with different portions of data.
- Each core perform identical operations of separate data

Difference between Data-level Parallelism & Instruction-level.

Data Level Parallelism	Instruction-level Parallelism
Performing same operation on multiple data elements simultaneously	Executing multiple independent instructions simultaneously
Parallelism among data items	Parallelism among instructions
Same instruction applied to different pieces of data	Different instructions executed at same time
Technique - SIMD (Single Instruction Multiple Data) & Vector processing	Technique - Pipelining, superscalar execution
requires independent data elements	Requires independent instructions
Suitable for image processing, matrix operations, graphics	Suitable for general purpose programs

Parallel Computer Architectures - Classifications

① Classification based on instruction and data stream
② Classification Based on structure of computers
③ Classification Based on how memory is accessed
④ Classification Based on ~~structure of~~ grain size



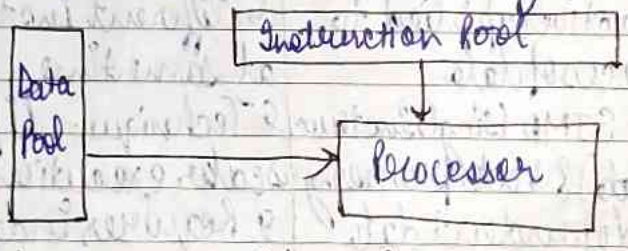
Date:
Page:

① Classification Based on instruction and data stream (Flynn's Taxonomy)

Instruction Stream \ Data Stream	Single	Multiple
Single	SISD Uniprocessors	SIMD Vector Processors Parallel Processing
Multiple	MISD Computers may be pipelined	MIMD Multi-computers Multi-Processors

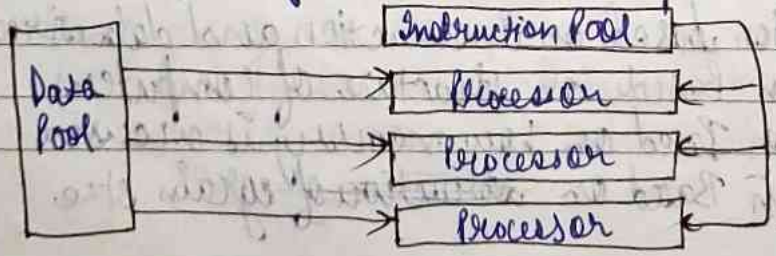
a) Single Instruction, ~~Multiple~~ Single Data Systems
- uniprocessor machine capable of executing single instruction, operating on a single data stream.
- instructions processed in a sequential manner.
- speed is limited (dependent) by rate at which computer can transfer information internally.

ex- IBM PCs.

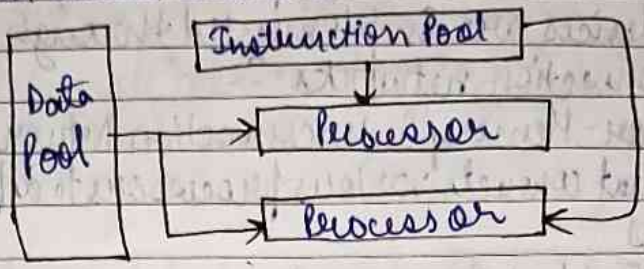


b) Single-Instruction, Multiple-Data System
- multi-processor machine capable of executing same instruction on all CPUs but operating on different data streams
- well suited for scientific computing since they involve lots of vector & matrix operations

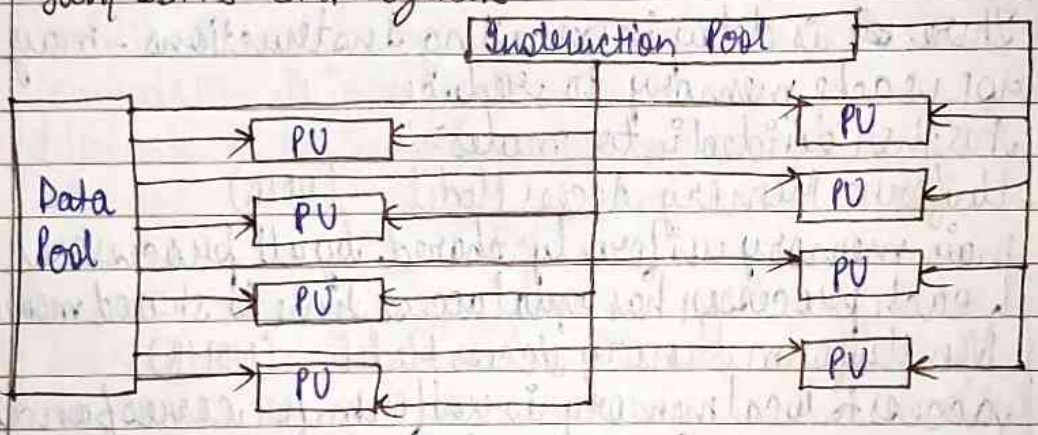
ex- Cray's vector processing machine



c) Multiple Instruction, Single-Data System
multiprocessor machine capable of executing different instructions on different processors but all of them operating on same dataset.



d) Multiple Instruction, Multiple Data System
multiprocessor machine capable of executing multiple instructions on multiple data sets
Each processor has separate instruction and data stream
work asynchronously.
eg- Sun/IBM's SMP system

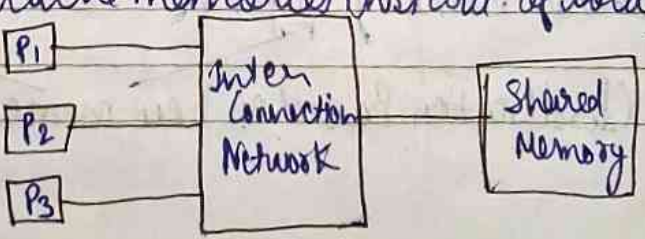


(2) Classification Based on structure
Tightly Coupled System/ Shared Memory System
Loosely Coupled System/ Distributed Memory System

(3) Classification Based on how memory is accessed

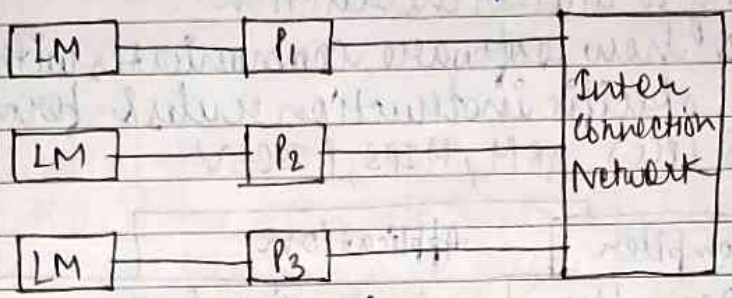
a) Shared Memory / Tightly Coupled System (which may have modules)
↳ multiple processors share global main memory
↳ processors have access to I/O devices
↳ Inter-communication b/w processors, memory and other devices are implemented through various interconnection networks:
(i) Processor-Memory Interconnection Network
↳ switch that connects various processors to different memory modules
(ii) Input-Output-Processor Interconnection Network
↳ all processors communicate with I/O chanel to interact with I/O device with prior permission of IOPIN
(iii) Interrupt Signal Interconnection Network
↳ synchronization b/w processor is implemented
↳ used when processor wants to send interrupt to another processor

↳ There is delay in executing instructions - may use cache memory to reduce
↳ further divided into modes:
(i) Uniform Memory Access Model (UMA)
↳ main memory uniformly shared by all processors & each processor has equal access time to shared memory
(ii) Non-Uniform Memory Access Model (NUMA)
↳ access to local memory is uniform for corresponding processor as it is attached to local memory
but if one reference is to local memory of some other remote processor → access non uniform
(iii) Cache Only Memory Access Model (COMA)
↳ In NUMA if we use cache memories instead of local, becomes COMA



Date:
Page:

b) Loosely Coupled / Distributed Memory System
- each processor has large local memory which is not shared by any other processor.
- such systems have multiple processors with own local memory and set of I/O devices.
- connected through message passing interconnection network (by passing messages).
- known as no-remote memory access systems



3) Based on Grain Size
- GrainSize (Granularity) - measure which determines how much computation is involved in a process
- determined by counting number of instructions in a program segment

Types
- Fine Grain: approx < 20 instruction
- Medium Grain: approx < 500 instructions
- Coarse Grain: approx >= 1000 instructions

- Degree of parallelism decreases with increase in granularity
- Every level demands communication & scheduling overhead.

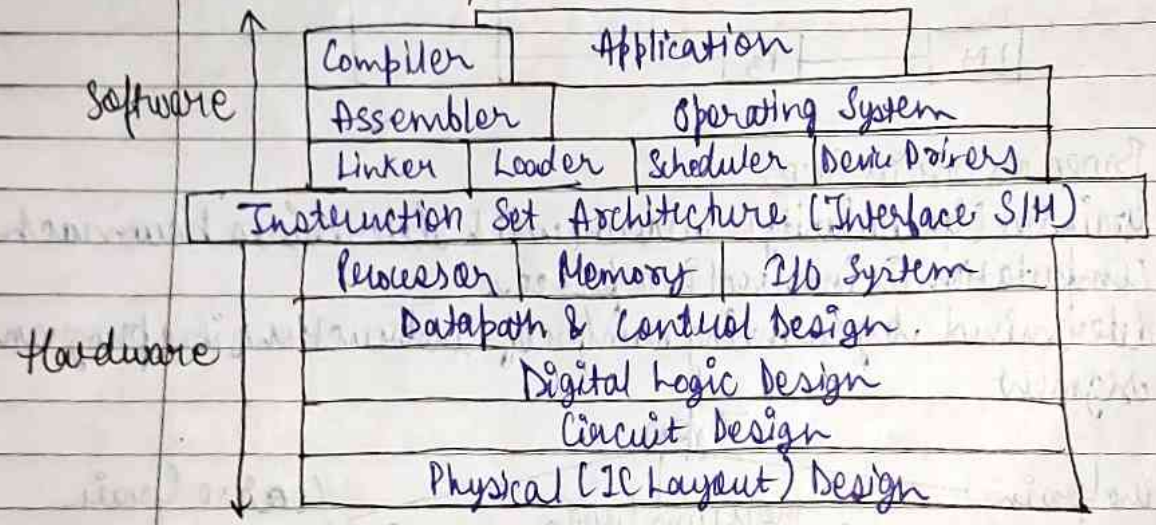
following are parallelism levels

- Instruction Level: lowest level
- Loop Level: for iterative loop level max compilers
- Procedure or Sub Program level: multiprogramming
- Program Level: independent programs, Time sharing

Grain Size	Parallelism Level
Fine Grain	Instruction or Loop Level
Medium Grain	Procedure or Subprogram Level
Coarse Grain	Program Level

Instruction Set Architecture

- Language of CPU that tells it what operations it can perform such as adding numbers, loading data or jumping to another instruction
- defines how software communicates with hardware through specific instruction rules & formats.
- ex- x86 (PCs), ARM, MIPS, RISC-V



Example - MIPS ISA:

- Types of Instructions
  Three:
  - Arithmetic/Logic Instructions (ADD, SUB, AND, etc)
  - Data Transfer Instruction - move data b/w memory & registers (LW and SW)
  - Branch & Jump Instructions - control execution flow
    BEQ, J (Jump)
    (Branch if equal)

Date:
Page:

Instruction Length
- 32-bit ISA. every instruction = 32 bits (4 bytes)
- simplifies design & makes it efficient for both hardware & compiler

Instruction Formats
- R-type - Arithmetic & Logic Operations
- I-type - Data Transfer & Conditional Branches
- J-type - Unconditional Jumps

Importance -
- Foundation of processor Design
- Instruction Execution Understanding
- Enables assembly language processing
- Impact on Performance Metrics
- Compatibility & Portability

Types of ISA -

RISC - Few, Simple Instructions for Speed
CISC - Many complex instructions
VLIW - (Very Long Instruction Word) - Run multiple operations in one instruction
EPIC - (Explicitly Parallel Instruction) - run things in parallel
Stack-based - Uses stack instead of registers.

Components of ISA → 1) Instruction Set - list of instructions
2) Instruction Formats - Bit level layout (Opcodes, operands, addr. modes)
3) Registers - numbers, types & roles (general purpose, floating point)
4) Data types - supported types (Int, float, vectors)
5) Memory architecture - addressing modes, virtual memory, etc.
6) Privilege levels - user mode, Kernel mode
7) Interrupts and Exceptions - async events & fault conditions

Date:
Page:

RISC vs CISC

RISC	CISC
- Reduced Instruction Set Computing	- Complex Instruction Set Computing
- Fixed-length instructions	- Variable length instructions
- usually 1 cycle per instruction	- Multiple cycles per instruction
- general purpose registers	- fewer registers
- simple and limited addressing modes	- Complex and Many addressing modes
- Larger code size	- Smaller code size
- Simple hardware, easier pipelining	- Complex hardware, harder pipelining
- Example - ARM, RISC-V, MIPS	Ex - x86, Intel 8086

Myopic View of Computer Architecture

- myopic view - having narrow or limited perspective
- In computer architecture, idea of looking a computer primarily as: 'A processor that executes instructions specified by the ISA'.
- Ignores or gives less attention to other aspects.

Myopic view:-

Program → ISA → Processor → Memory

- computer performance does not depend only on instruction set.
- A processor may have same ISA but completely different implementations

example - Two processors → both x86-64 but one faster because: pipeline depth, cache size, branch prediction, clock frequency, memory system may vary

Processor Speed

- number of cycles CPU can execute per second (clock frequency)
- measured in Hz, MHz, GHz
- how quickly processor can execute instructions.

Example- 2 GHz = 2×10^9 cycles/second

$$\text{Clock Cycle Time} = \frac{1}{\text{Clock Rate}}$$

High clock rate → ~~too~~ short clock cycle time

- clock speed is one of the factor affecting performance but it is not complete measure of processor performance

Power Consumption

- Power - rate at which electrical energy is consumed by processor or computer system
- measured in Watts (W)

$$\text{Power} = \frac{\text{Energy}}{\text{Time}}$$

- Dynamic Power - consumed when transistors switch b/w states

$$\text{Dynamic} = \alpha C V^2 f$$

where

α = switching activity
C = capacitance
V = voltage
f = frequency

∴ Increase in frequency & voltage → increase power consumption

- Static Power - consumed when transistors are not actively switching

↳ associated with : • Leakage Current
& Transistor characteristics

Total Power = Pdynamic + Pstatic

Importance : High power causes :
- Heat generation
- Cooling requirements
- Reduced battery life
- Thermal throttling
- Reliability issues

↳ Power & Performance Trade off
↳ Increasing processor speed can increase performance but may also increase power consumption

Example -

Frequency ↑ => Performance ↑

But generally

Frequency ↑ => Power ↑

Similarly,

Voltage ↑ => Power ↑ ($P \propto V^2$)

Solution : Instead of only increasing clock frequency, modern processors use :
- multiple cores
- Parallel processing
- Dynamic voltage & frequency scaling

Fabrication Yield

↳ percentage of manufactured chips that function correctly and meet required specifications

ex- 1000 chips - 900 work Yield = $\frac{90}{100} \times 100 = 90\%$

Date:
Page:

$$\text{Yield} = \frac{\text{Number of Good Chips}}{\text{Total number of manufactured chips}} \times 100$$

→ Factors affecting fabrication yield → • manufacturing defects
- chip size (chip size ↑ → Probability of defect ↑)
- Manufacturing technology
- Number of transistors
- Process quality

- High yield means - Low manufacturing cost
  - Less material wastage
  - More usable chips
  - Better production efficiency
- High Chip Complexity → High fabrication yield difficult

Trends in Technology

- Integrated Circuit logic technology
- Known as Moore's law
- Traditional growth in transistor count on a chip every 18-24 months
- Semiconductor DRAM (Dynamic Random Access Memory)
- foundation of main memory
- quadrupling every 3 years in past (no 32GB DRAM)
- Semiconductor Flash (electrically erasable programmable ROM)
- non volatile → standard storage device in PMDs.
- doubles every 2 years - capacity per flash chip
- Magnetic Disk Technology
- add more platters at same areal density
- cheaper than Flash
- central to server and warehouse scale storage

Date:
Page:

- Network Technology
  - Network performance depends on performance of switches & transmission system

Trends in cost - Impact of time, volume & commoditization

- Time
  - Manufacturing cost ↓ over time due to learning curve
  - Measured through yield.
  - Prices of DRAM & microprocessors ↓ over time
- Volume
  - Higher production volume speeds learning curve, cost ↓
  - improve purchasing & manufacturing efficiency. Cost ↓
  - spreads development cost over more products, cost ↓
- Commoditization
  - standardized products sold by multiple vendors
  - High competition reduces gap b/w cost & selling price
  - Reduces overall cost

Cost of Integrated circuit

- cost ↓ exponentially
- basic process of silicon manufacture unchanged
- wafer is still tested and chopped into dies.

$$\text{Cost of integrated circuit} = \frac{\text{Cost of die} + \text{Cost of testing die} + \text{Cost of packaging \& final test}}{\text{Final test yield}}$$

$$\text{Cost of die} = \frac{\text{Cost of wafer}}{\text{Dies per wafer} \times \text{Die yield}}$$

$$\text{Dies per wafer} = \frac{\pi \times (\text{Wafer Diameter}/2)^2}{\text{Die area}} - \frac{\pi \times \text{wafer diameter}}{\sqrt{2 \times \text{Die area}}}$$

Date:
Page:

$$\text{Die yield} = \text{Wafer yield} \times \frac{1}{(1 + \text{Defects per unit area} \times \text{Die area})^N}$$

Dependability

- ability of computer system to deliver correct and reliable service over a period of time.
- Major aspects :
  - Reliability - probability that system operates correctly for a specific period
  - Availability - system is operational & ready to provide service
  - Maintainability - ease & speed with which it can be repaired
  - Safety - avoid unacceptable damage
  - Security - protect against unauthorized access

Effect of Energy, Power and Cost on Dependability :

1) Power — Power ↑ → Heat ↑
   - High power consumption → more heat
   - Excessive heat cause - component degradation, thermal failures, system instability, reduced hardware lifetime.
   - Reduces reliability & availability
2) Energy
   - important in battery-powered system (laptop, smartphones)
   - High energy consumption — drain battery faster, system can shut down and reduces operational time
   - Energy Consumption ↑ → Battery Life ↓ → Availability ↓
3) Cost
   - ↑ dependability requires additional hardware → Cost ↑
   - Example - redundant power supplies, ECC memory, Backup, fault-tolerant processors → Cost ↑ → Dependability ↑

Availability

- measures fraction of time for which system is operational and accessible

$$\text{availability} = \frac{\text{Uptime}}{\text{Uptime} + \text{Downtime}}$$

$$\boxed{\text{Availability} = \frac{MTBF}{MTBF + MTTR}}$$

MTBF - Mean Time Between Failures
MTTR - Mean Time To Repair

ex- MTBF = 200 hours MTTR = 2 hours

$$\text{availability} = \frac{200}{200+2} = \frac{200}{202} = 0.990$$

In percentage = 0.9901 × 100

availability = 99.01%

Performance Metrics & Evaluation

Measuring Performance

- Performance Metrics gives a quantitative way to evaluate and compare computer systems
- Some Performance Metrics:

1) Execution Time
- total time a system takes to complete specific task or program
- measured in seconds or clock cycles
- driven by instruction count, average cycles per instruction (CPI) and clock speed

Execution Time:

$$\boxed{\text{CPU Time} = \text{Instruction Count} \times CPI \times \text{Clock Cycle Time}}$$

Execution Time/ CPU Time = $\frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock Rate}}$

2) Throughput
- number of tasks or operations completed per unit of time
- units – Instruction per second, (IPS)
  Floating point operations per second (FLOPS)
  Transactions per second (TPS)
- reflects system's processing capacity and relevant for server and batch-processing workloads

3) Latency
- time delay between initiating a request and receiving response
- measured in seconds or clock cycles
- captures how responsive a system is to individual requests
- system can have high throughput also high latency

4) Speedup
- measures how fast system becomes after improvement

$$\text{Speedup} = \frac{\text{Old Execution Time}}{\text{New Execution Time}}$$

Factors influencing performance
1) Instruction Set Architecture (RISC & CISC)
2) Pipline Design (multiple instructions overlap)
3) Cache hierarchy (Latency)
4) Memory system architecture (movement b/w CPU & Memory)
5) Instruction level parallelism
6) Thread Level parallelism
7) Clock Frequency
8) Power Constraints

Benchmark standards

- measures system performance using standardized workloads, enabling objective comparisons between architectures or configurations
- choice of benchmark matters because workloads stress different parts of the system

• Synthetic benchmarks

- artificial workloads designed to stress specific subsystems
- LINPACK – measures floating point performance
- STREAM – measures sustainable memory bandwidth
- useful for isolating specific capabilities but don't necessarily reflect real application performance

• Application-specific benchmarks

- uses real-world programs representative of actual usage scenarios
- give more realistic performance estimates for targeted domains.

- SPEC CPU – covers general purpose integer & floating point computing
- TPC-C – measures online transaction processing
- MLPerf – evaluates ML training & inference performance

- preffered for architectural comparisons because they capture interactions b/w ISA, memory hierarchy, parallelism.

Iron Law of Performance

↳ To disentangle the parameters that affect execution time of a program, we break them down into "iron law" of performance

$$\frac{\text{time}}{\text{program}} = \frac{\text{instructions}}{\text{program}} \times \frac{\text{cycles}}{\text{instructions}} \times \frac{\text{time}}{\text{cycles}}$$

↳ expands execution time (time/program) into product of instruction count, cycles per instruction & clock period

$$\text{Execution Time} = IC \times CPI \times CCT \quad (\text{s/ms/ns etc})$$

IC - Instruction count CCT - clock cycle time

CPI - cycles per instruction

- Instruction/program - no. of instructions present in benchmark that we are comparing processors on
- Cycles/instructions - average of all instructions executed in program.
- Clock period (time/cycles) - time taken to finish one clock cycle completely depends on hardware

Instruction Throughput - Instructions completed per unit of time.

$$\text{Throughput} = \frac{\text{instructions}}{\text{time}} = \frac{\text{instructions}}{\text{cycles}} \times \frac{\text{cycles}}{\text{time}} = \frac{f}{CPI}$$

Amdahl's law

↳ proposed by Gene Amdahl

↳ Explains theoretical speedup of program when part of it is improved or parallelized.

↳ used in parallel computing to predict benefits of using multiple processors

↳ implies we have to speed up most common causes in order to have significant impact on overall speedup.

Date:
Page:

- Speedup - performance improvement gained
  $S = \frac{\text{Old Execution Time}}{\text{New Execution Time}}$
- fraction enhanced (P) - proportion of program that can be parallelized ($0 < P \leq 1$) / improved
- Number of processors (N) - no. of parallel units used for execution

$$\text{Speedup} = \frac{1}{(1-P) + \frac{P}{S}} \quad \text{OR} \quad S = \frac{1}{(1-P) + \frac{P}{N}}$$

$1-P$ = portion that can not be improved / parallelized
S = speedup of improved portion

Example- 40% of program is parallelizable
using 4 processors speedup?

$P = 0.4$ (40%)
$N/S = 4$

$$\text{Speedup} = \frac{1}{(1-0.4) + \frac{0.4}{4}} = \frac{1}{0.6+0.1} = 1.43$$

Advantages
- provides clear upper bound on performance
- Helps identify bottlenecks in program
- Useful in guiding hardware / software design decisions

Disadvantages
- assumes sequential part is fixed
- assumes processors are identical
- Ignores real world factors like comm", synchronization, load balancing overhead.

Date:
Page:

Amdahl's law

- we don't want to mess up with the uncommon cases too badly
- overall performance is limited by the portion of system that is not improved

Example- we want speedup of 2 on 90% of execution time & another speedup of 0.1 on rest. Overall speedup

$$= \frac{1}{\frac{10\%}{0.1} + \frac{90\%}{2}} = 0.7$$

Ans: suppose original execution time = 100 s

Common case (90%) = 90s → speedup = 2x → New time = $\frac{90}{2}$ = 45

Uncommon (10%) = 10s → speedup = 0.1x → New time = $\frac{10}{0.1}$ = 100 s

Total = 45 + 100 = 145 s

Uncommon case makes a difference. Despite 2x increase in common case, system is slower. (100 − 145 s)